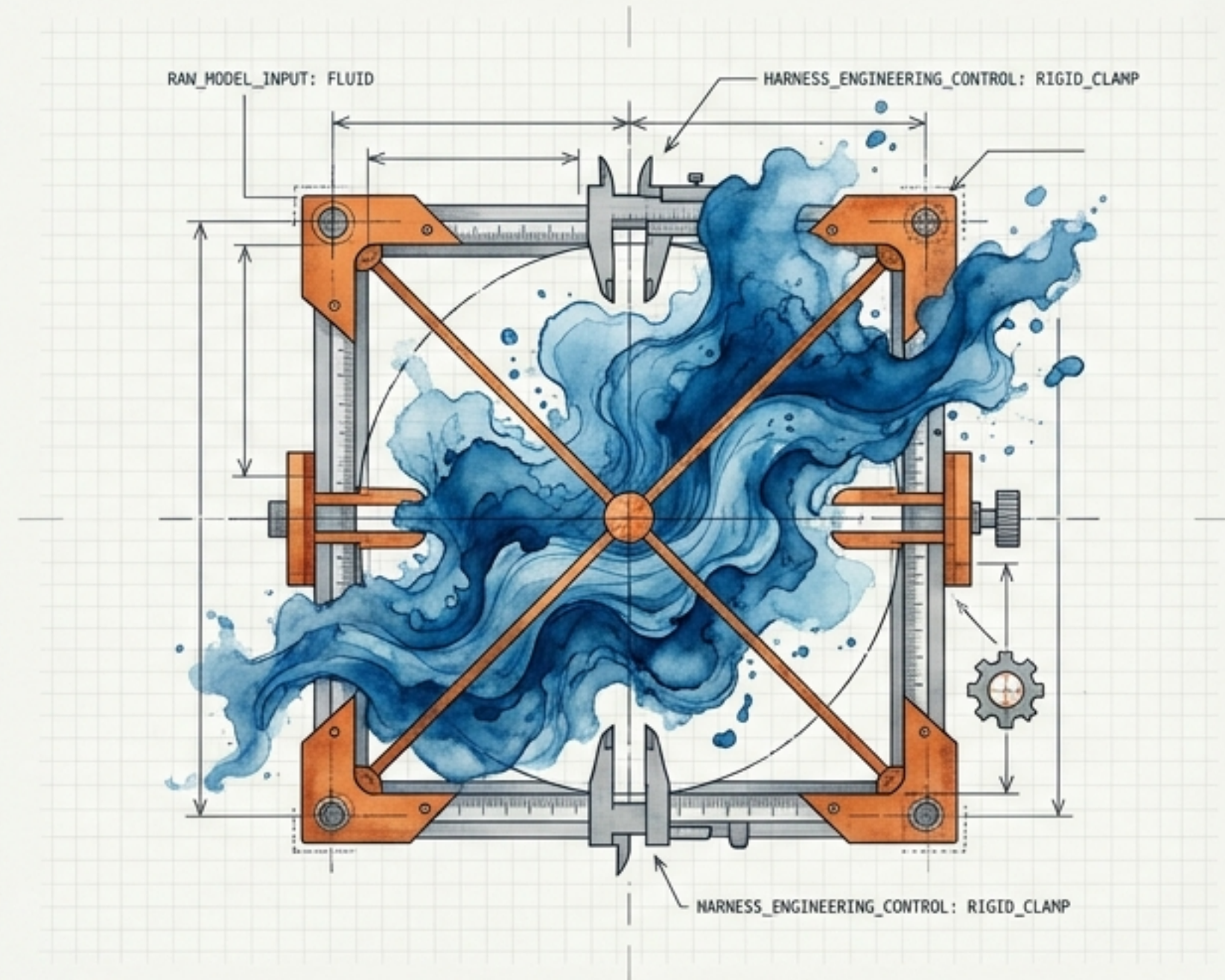




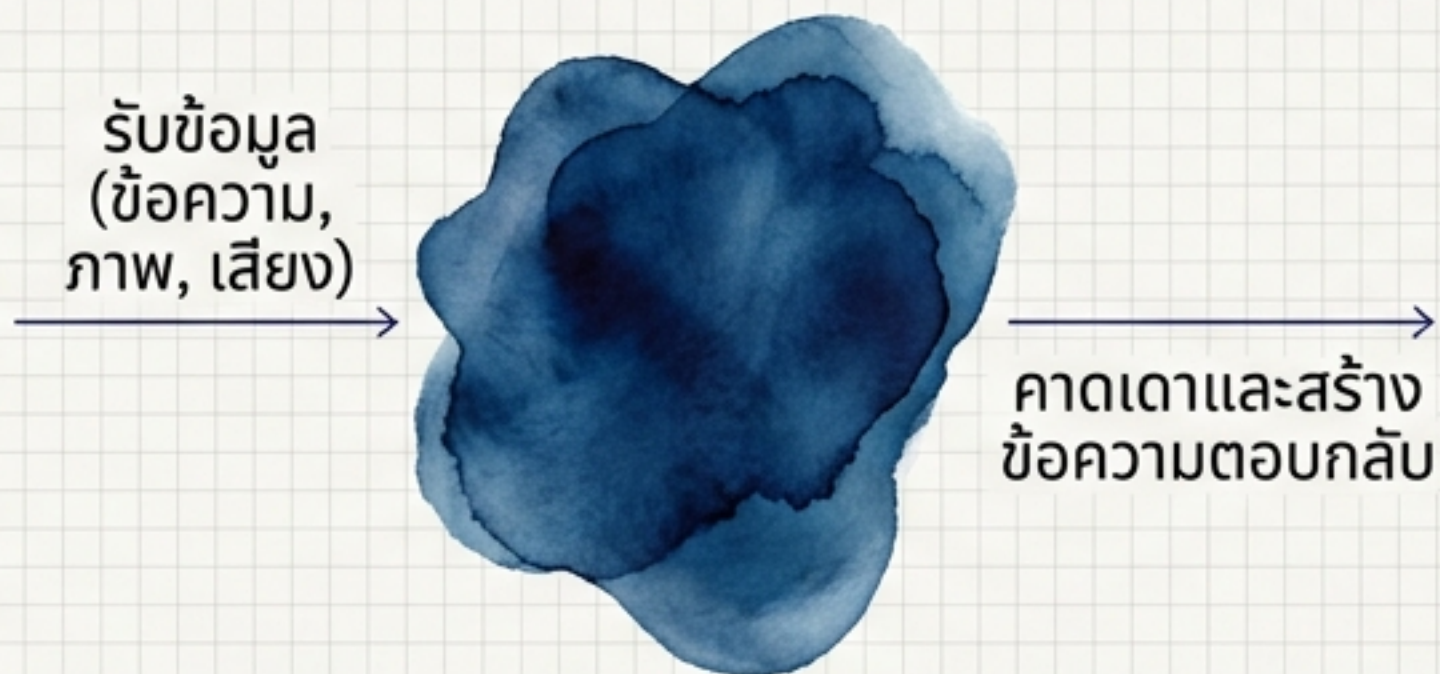
สถาปัตยกรรมแห่งระบบอัตโนมัติ: คู่มือฉบับสมบูรณ์สำหรับ Harness Engineering

เปลี่ยนจากโมเดลเชทบอกพื้นฐาน สู่ระบบ AI Agent ที่ทำงานได้จริงในระดับโปรดัคชัน

โมเดลที่ชาญฉลาด ไม่ใช่ Agent ที่สมบูรณ์

$$[Agent] = [Model] + [Harness]$$

ความสามารถพื้นฐานของ LLM



สิ่งที่ LLM ทำเองไม่ได้ในโลกความเป็นจริง



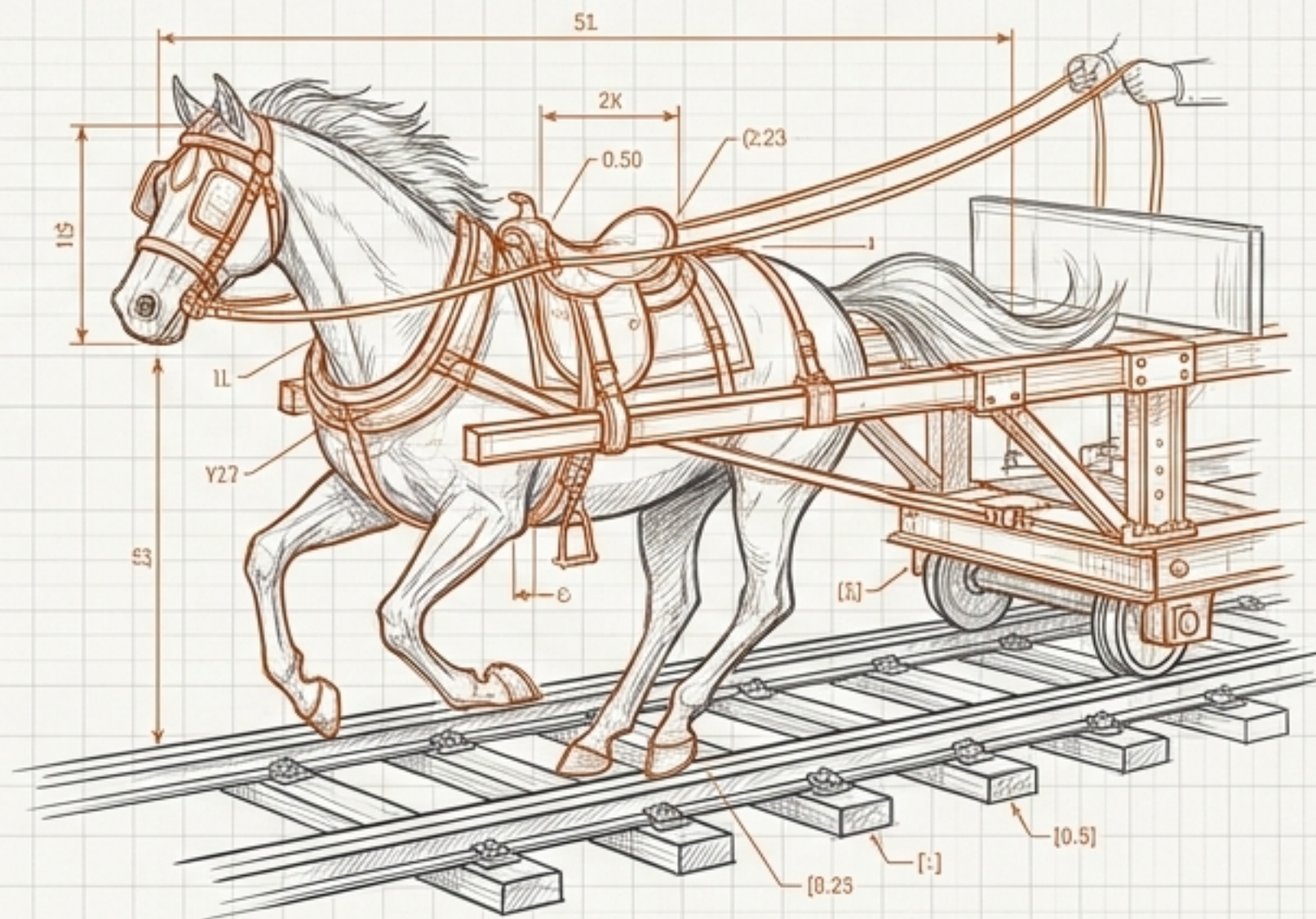
ระบบ Harness คือการสร้างโครงสร้างพื้นฐานล้อมรอบความฉลาดของโมเดล เพื่อเปลี่ยนมันให้เป็น 'เครื่องยนต์' ที่ทำงานได้จริง

ปรัชญาของ Harness Engineering: ความฉลาดที่ปราศจากการควบคุมคือความเสี่ยง

[Model] คือม้าป่าที่ทรงพลัง (The Horse)
รวดเร็ว ฉลาด แต่ไม่มีทิศทาง

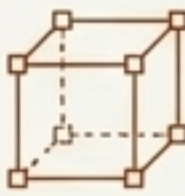


[Harness] คือสายบังเหียนและราง (The Reins & Tracks)
[Harness Engineer] คือผู้บังคับที่ออกแบบระบบควบคุม



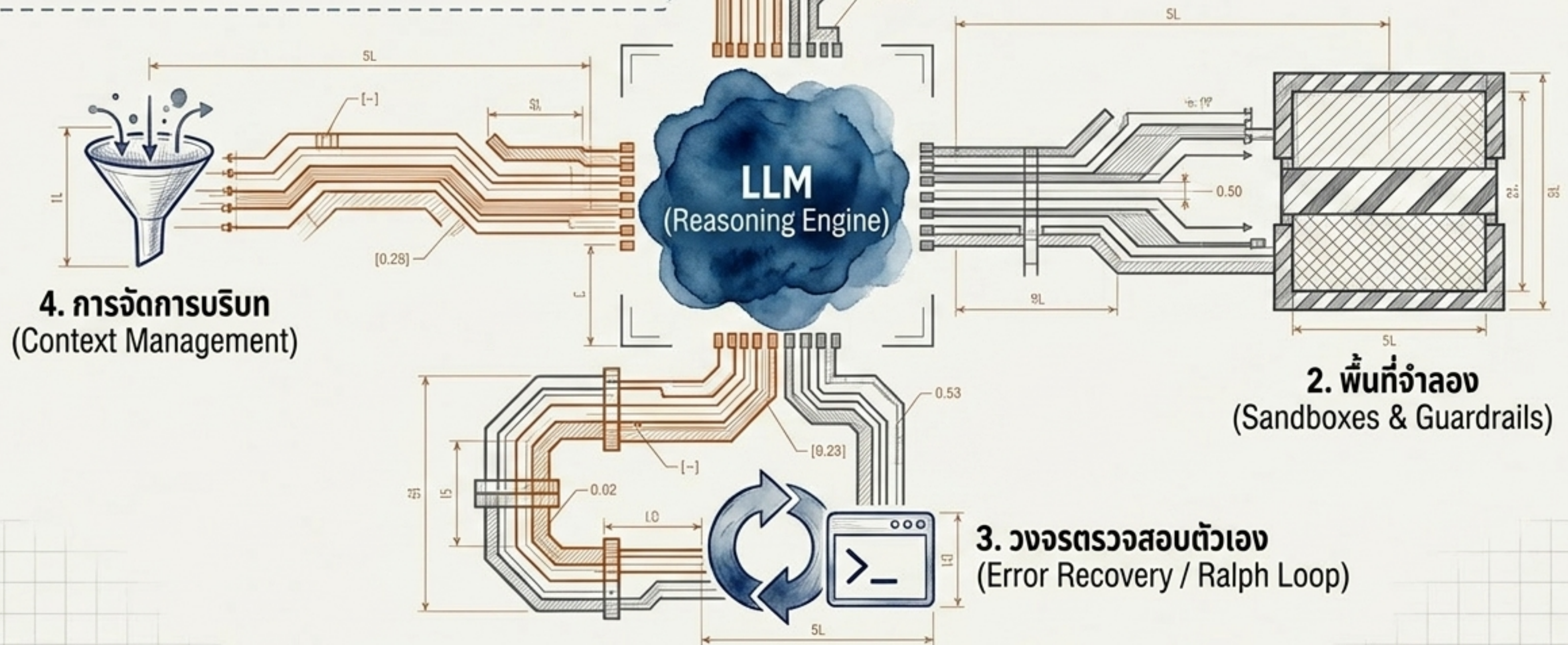
การมีแค่โมเดลที่ฉลาดที่สุด ไม่ได้การันตีความสำเร็จ
หากระบบควบคุมไม่สามารถจัดการกับความผิดพลาด ทิศทาง และขอบเขตการทำงานได้

การวินิจฉัยระบบ: ทำไมเราถึงต้องการ Harness?

มิติการทำงาน (Dimension)	โมเดลเปล่า (Naked Model) 	Agent ที่มีระบบ Harness 
การจดจำ (State & Memory)	ภาวะความจำเสื่อม (Amnesia): ลืมบริบททั้งหมดเมื่อเริ่มเซสชันใหม่ หรือหลุดจาก Context Window	ความจำถาวร (Durable State): บันทึกสถานะการทำงานผ่านระบบไฟล์ (เช่น AGENTS.md) และเรียกใช้เมื่อต้องการ
ความแม่นยำ (Accuracy)	มั่นใจแบบผิดๆ (Confident Hallucinations): สร้างคำตอบที่ดูสมจริงแต่ใช้งานไม่ได้ โดยไม่รู้ตัว	ตรวจสอบตัวเอง (Self-Verification): รันเทสต์ ถึง Error Log และแก้ไขโค้ดใหม่ จนกว่าจะผ่าน
การลงมือทำ (Execution)	แค่สร้างข้อความ (Text Generation): ส่งโค้ดให้มนุษย์นำไปรันต่อ	ทำงานอิสระ (Autonomous Action): รัน Bash Command, จัดการ Dependency และทดสอบระบบตัวเองใน Sandbox
ความปลอดภัย (Safety)	ไร้ขีดจำกัด (Unbounded Liability): สามารถปล่อยคำสั่งอันตรายหรือ ทำข้อมูลรั่วไหล	มีกลไกป้องกัน (Guardrails): กำหนดสิทธิ์, ตรวจสอบลูปอนันต์ และมี จุดตรวจสอบโดยมนุษย์

สถาปัตยกรรม 5 เสาหลักของ Harness

แผงวงจรหลักที่เชื่อมต่อกับหน่วยประมวลผล (LLM) เข้ากับทรัพยากรเพื่อให้เกิดการทำงานแบบอัตโนมัติ



เสาหลักที่ 1: การจัดการสถานะและเครื่องมือ

'ความสามารถของ Agent ถูกกำหนดโดยขอบเขตเครื่องมือที่มันเข้าถึงได้'

พื้นที่ทำงานถาวร (Workspace)

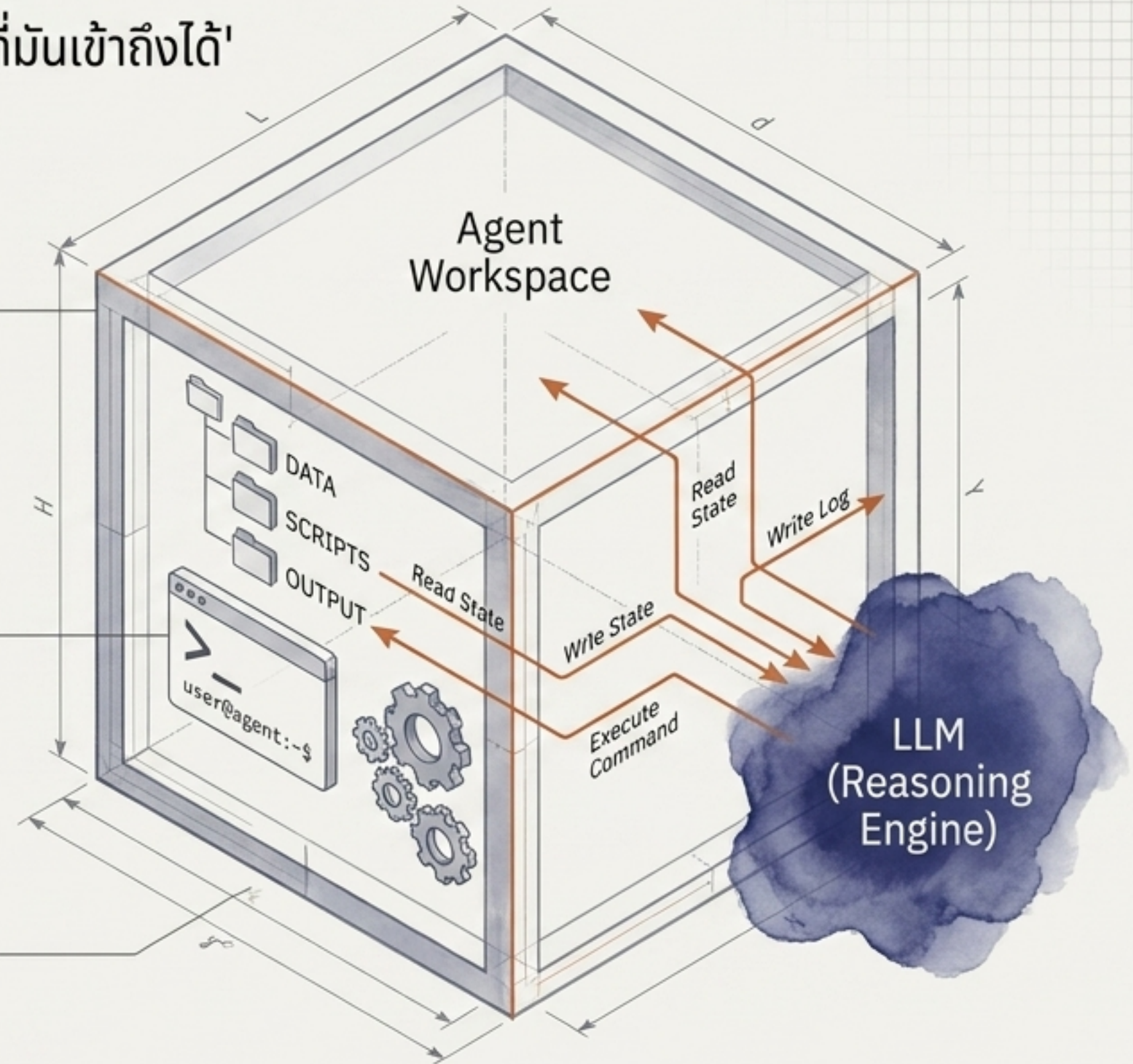
โมเดลทำงานกับข้อมูลผ่านระบบไฟล์
ทำให้สามารถพักข้อมูลและกลับมาทำต่อได้
โดยไม่ต้องอัดทุกอย่างลงใน Context Window

เครื่องมืออเนกประสงค์ (Bash + Code)

การให้สิทธิ์รัน Bash คือการ 'มอบคอมพิวเตอร์ให้โมเดล' เพื่อให้มันค้นหาวิธีแก้ปัญหาและสร้าง
เครื่องมือขึ้นมาใช้เองแบบ **On-the-fly**

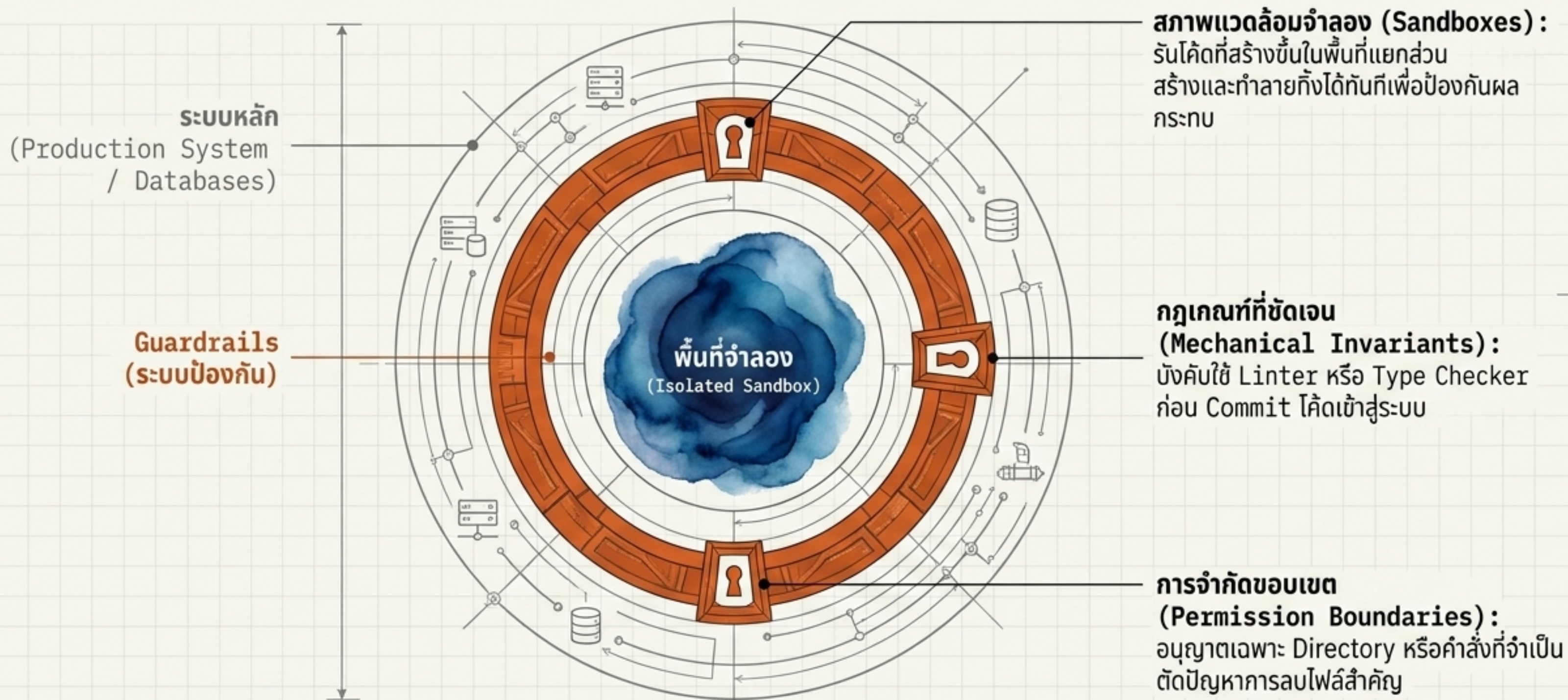
การทำงานร่วมกัน (Collaboration)

ระบบไฟล์และ **Git** คือพื้นที่ประสานงานระหว่าง
Agent หลายตัวและมนุษย์



เสาหลักที่ 2: ขอบเขตและพื้นที่ปลอดภัย

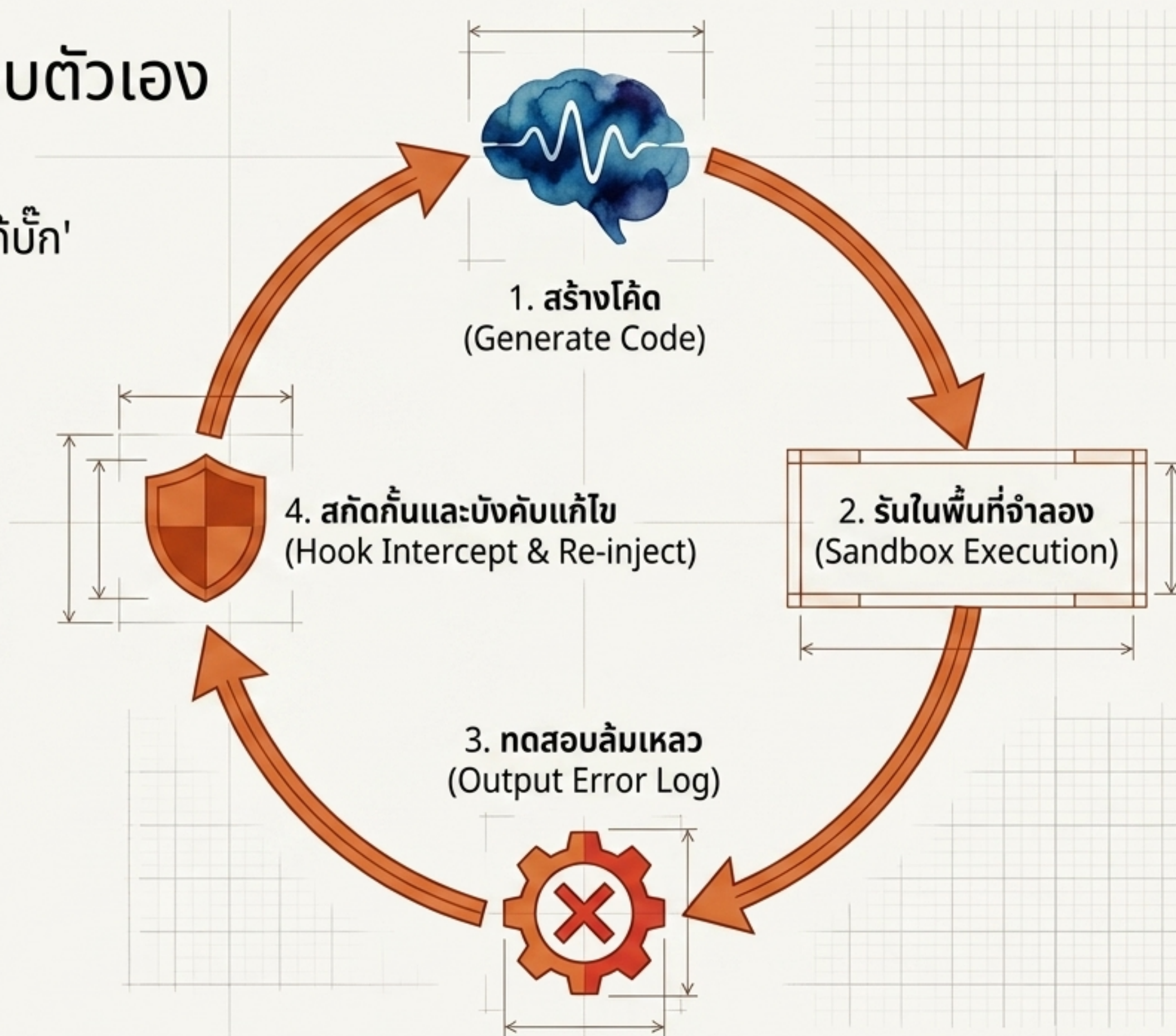
ข้อจำกัดที่ชัดเจน ช่วยเพิ่มประสิทธิภาพ ไม่ใช่แค่ความปลอดภัย



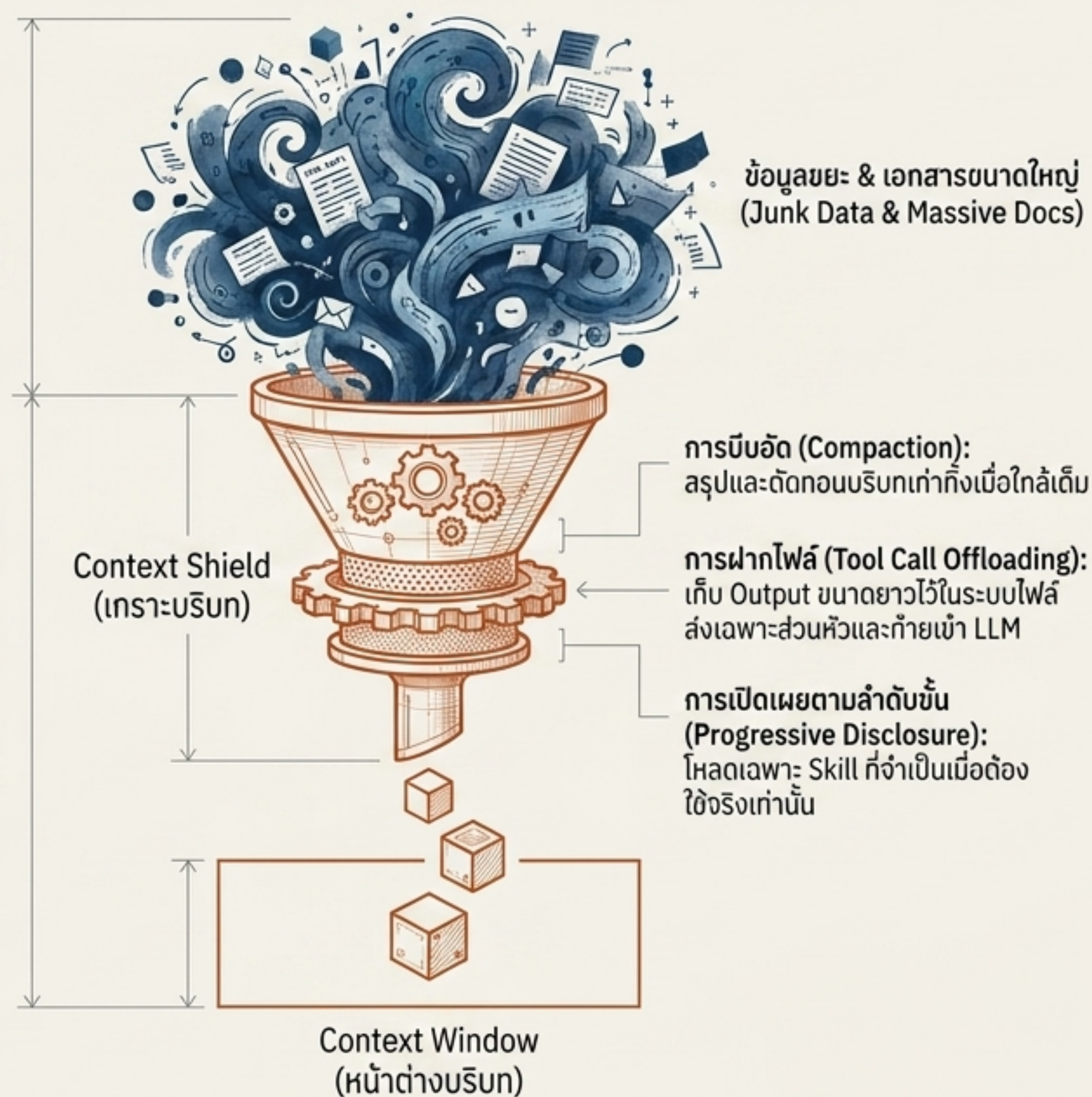
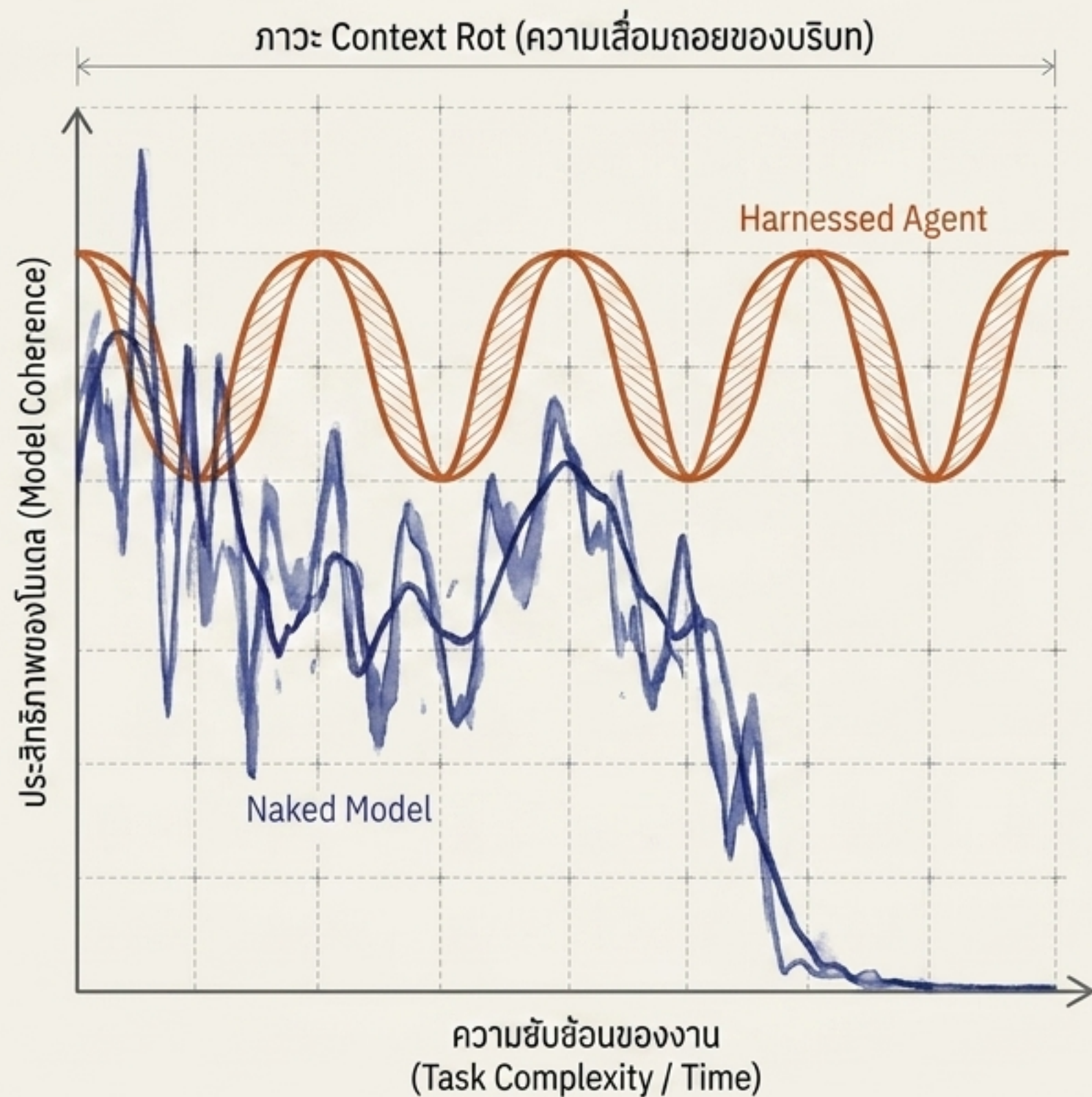
เสาหลักที่ 3: วงจรตรวจสอบตัวเอง (The Ralph Loop)

เปลี่ยนการ 'เดา' เป็นการ 'รัน เทสต์ และแก้บัก'

- **ดักจับความล้มเหลว:** เมื่อโค้ดรันไม่ผ่าน Harness จะทำงานโดยสกัดกั้นไม่ให้โมเดลหยุดทำงาน
- **ป้อนข้อผิดพลาดกลับ:** นำ Error Log ป้อนกลับเข้าไปใน Context ใหม่ที่สะอาด
- **ผลลัพธ์:** บังคับให้ Agent วิเคราะห์ Error และเขียนโค้ดแก้บักจนกว่า Test Suite จะผ่าน (Self-Verification)

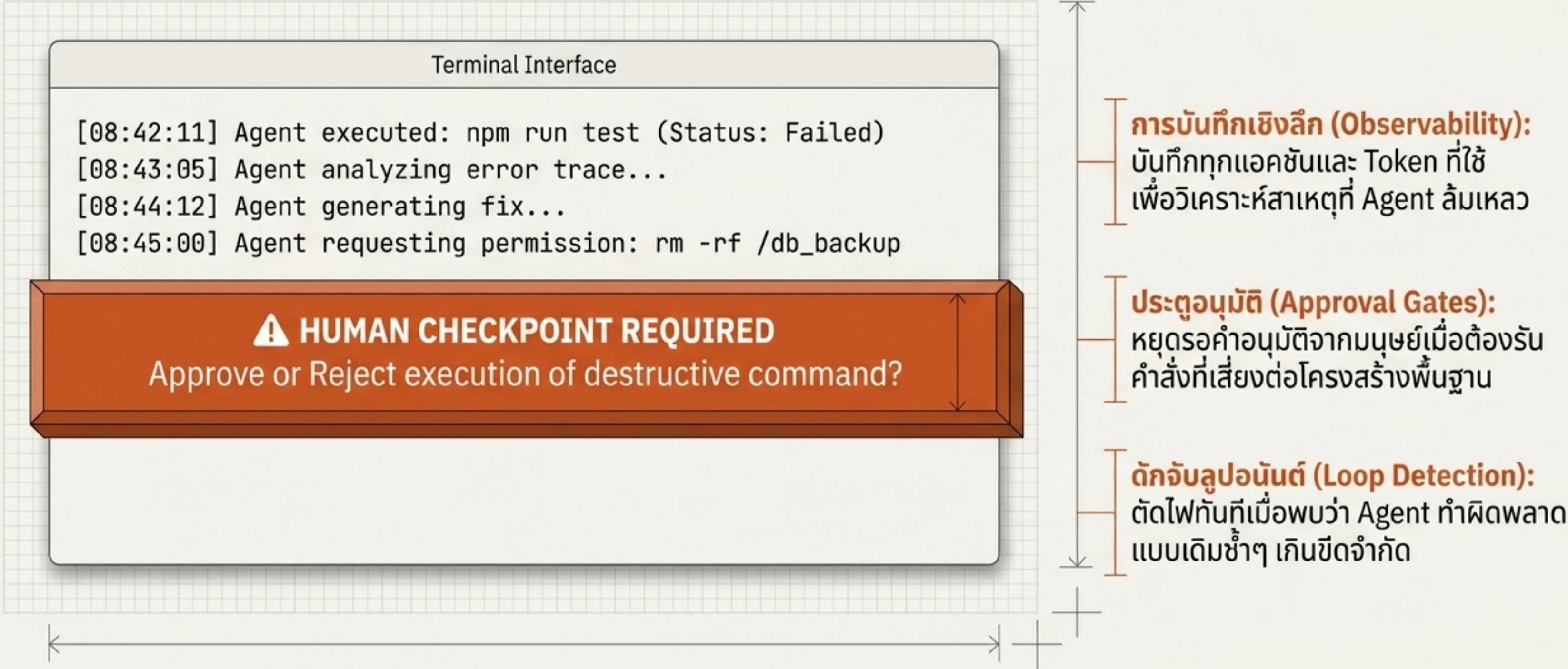


เสาหลักที่ 4: การจัดการหน่วยความจำและ Context Rot



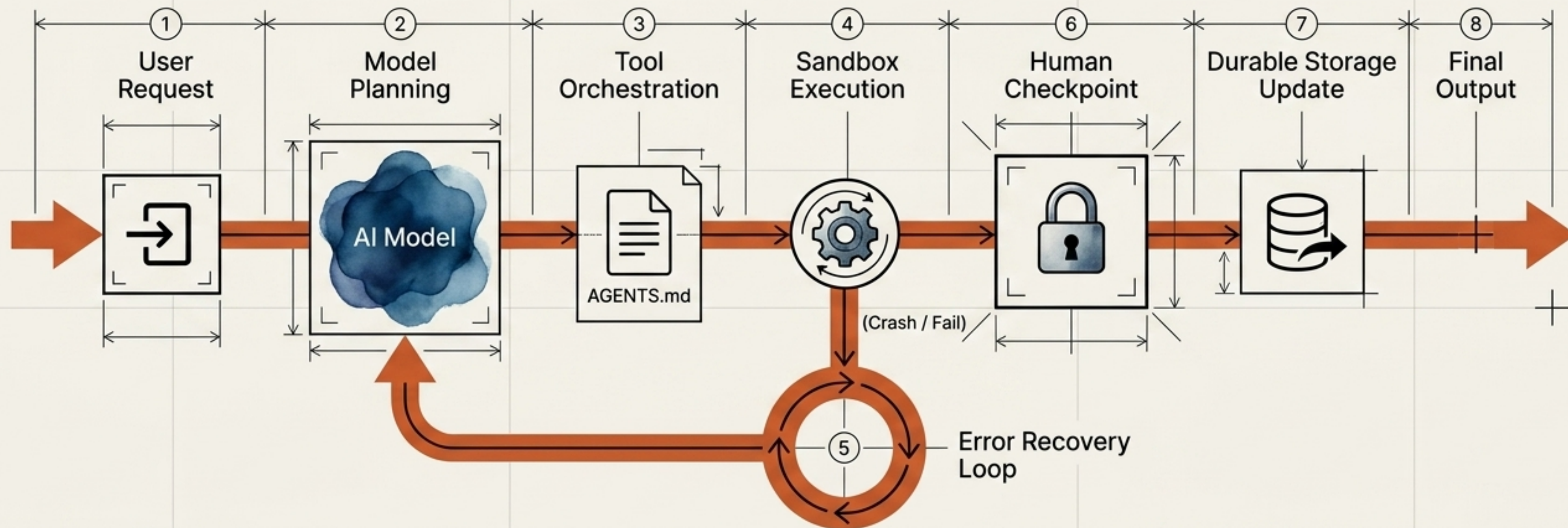
เสาหลักที่ 5: การสังเกตการณ์และจุดตรวจสอบโดยมนุษย์

ความเป็นอัตโนมัติเต็มรูปแบบไม่เหมาะกับทุกงาน คุณไม่สามารถปรับปรุงสิ่งที่คุณมองไม่เห็นได้



สังเคราะห์: วงจรชีวิตของงานอัตโนมัติ

การทำงานประสานกันของทุกองค์ประกอบเมื่อรับมือกับความซับซ้อน



เมื่อ User มอบหมายงาน ระบบจะไม่เพียงแค่ส่ง Prompt แล้วจบ แต่ทำงานเป็น "เครื่องยนต์แบบวัฏจักร": วางแผน โหลดกฎจากระบบไฟล์ รันโค้ดใน Sandbox วนลูปแก้บั๊กเอง ร้องขออนุมัติ และอัปเดตสถานะ

เมทริกซ์สายงานวิศวกรรม AI:

Harness Engineering คืออะไร?

มิติ (Dimensions)	Prompt Engineering	MLOps	DevOps	Harness Engineering
เป้าหมายหลัก	สร้างข้อความ Input ที่ดีที่สุดสำหรับ 1 ครั้ง	บริหารจัดการวงจรชีวิตของ Model Weights	ระบบอัตโนมัติในการส่งมอบซอฟต์แวร์	ออกแบบระบบที่โอบล้อมควบคุม AI Agents
ปัญหาที่แก้	การตีความผิดพลาดของโมเดล	ประสิทธิภาพโมเดลและ Infrastructure	คอขวดในการ Deploy	การสูญเสียบริบท, อาการหลอน, การทำงานระยะยาว
สิ่งที่สร้าง (Artifacts)	System Prompts	Training Pipelines	CI/CD Pipelines	AGENTS.md, Sandboxes, Hooks, Loops
ผลลัพธ์ (Outcome)	คำตอบที่ตรงจุดมากขึ้น	โมเดลที่ทำงานได้เสถียร	โค้ดถูกปล่อยขึ้นระบบอย่างปลอดภัย	Agent ที่รับผิดชอบงานได้เองจนจบ

กรณีศึกษา: การใช้งานจริงในระดับโลก

OpenAI Codex

- ใช้ AGENTS.md เป็นคำสั่งอ่านได้ด้วยเครื่อง เพื่อกำหนดทิศทาง Agent
- สร้างโค้ดมากกว่า 1 ล้านบรรทัดโดยไม่ต้องใช้มนุษย์เขียน
- รัน CI ที่บังคับใช้กฎสถาปัตยกรรม (Mechanical Invariants) แบบ 100%

Anthropic Claude Code

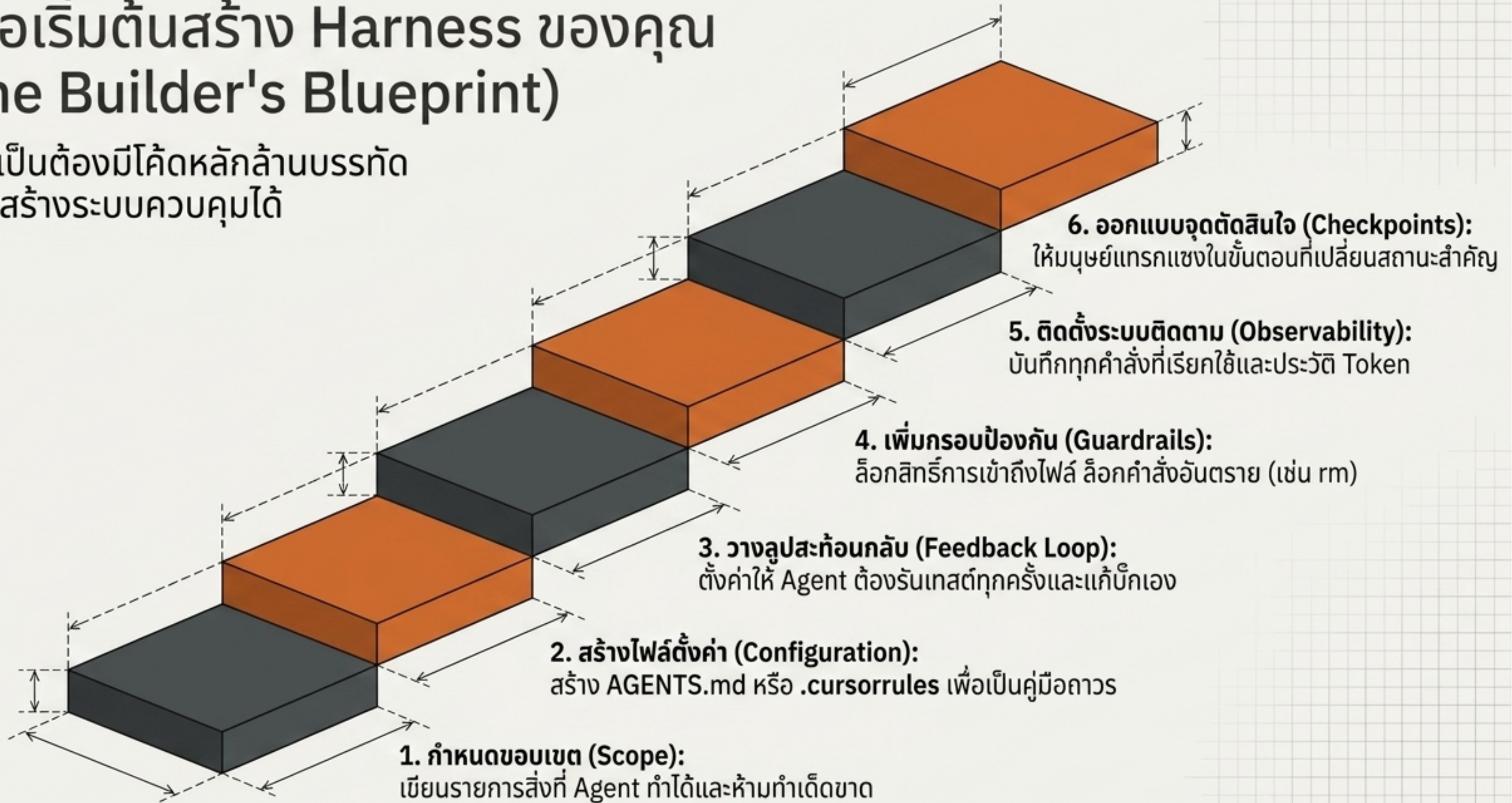
- ใช้ระบบ Permissions ที่อ่านได้อย่างเดียวเป็นค่าเริ่มต้นจนกว่าจะได้รับสิทธิ์
- ใช้ระบบ Hooks เพื่อสแกนความปลอดภัยและรัน Linter ก่อน Commit
- ใช้ CLAUDE.md เพื่อเก็บรักษากฎของโปรเจกต์ให้คงอยู่ข้ามเซสชัน

Cursor (IDE)

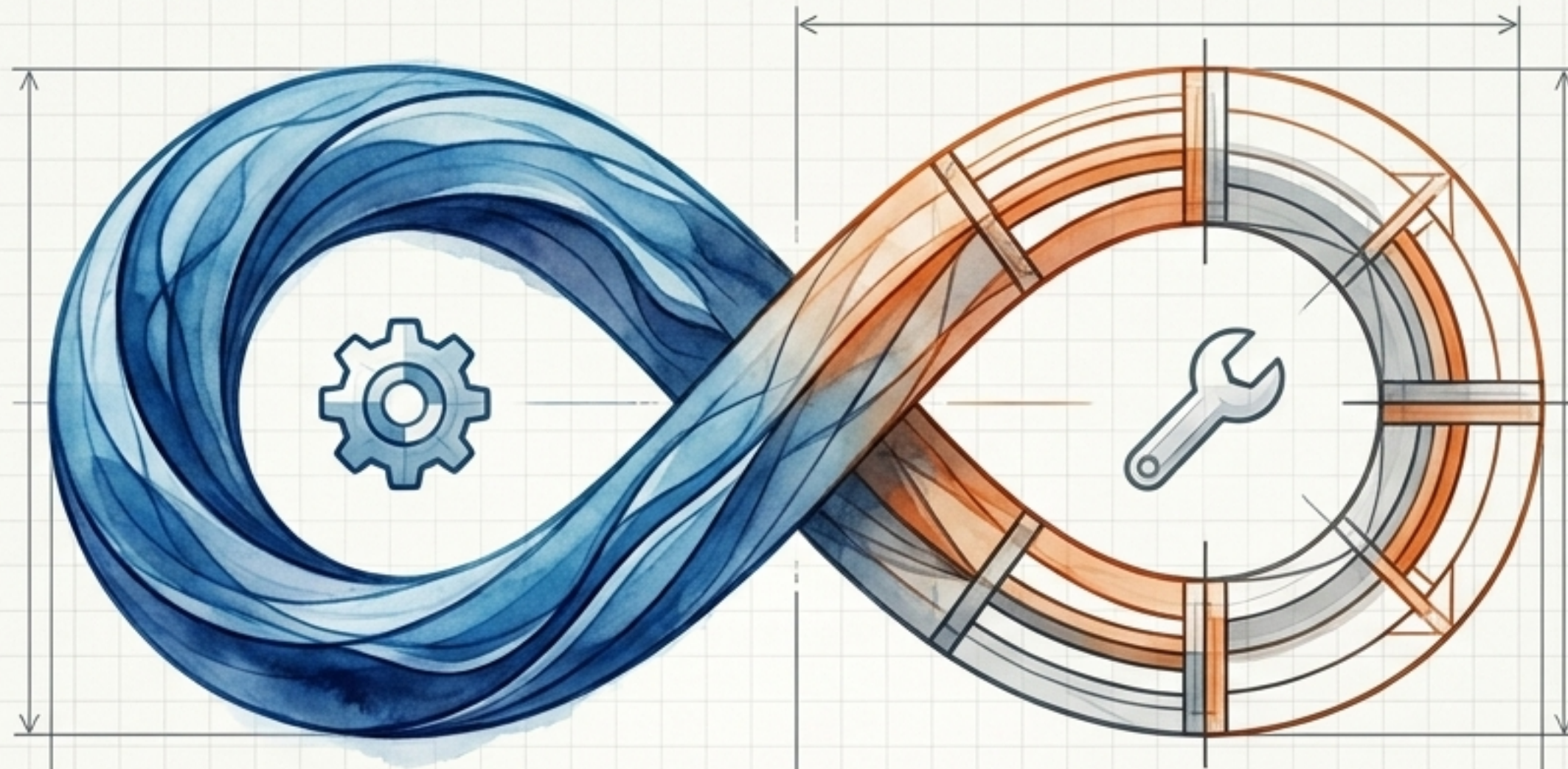
- ใช้ไฟล์ .cursor/rules เป็นระบบ Harness ที่ทำงานตลอดเวลา
- ตรวจจับคู่ป้อนันต์ได้อัตโนมัติ
- ปรับ Prompt แบบเจาะจงโมเดลตามบริบทของโค้ดเบส

คู่มือเริ่มต้นสร้าง Harness ของคุณ (The Builder's Blueprint)

ไม่จำเป็นต้องมีโค้ดหลักล้านบรรทัด
ก็เริ่มสร้างระบบควบคุมได้



อนาคตของการพัฒนาระบบ AI: วิวัฒนาการร่วมกัน (Co-evolution)



52.8% → 66.5%

การเพิ่มความสามารถของ Agent บน Terminal Bench 2.0
โดยไม่ต้องเปลี่ยนโมเดล... อาศัยแค่การเปลี่ยนระบบ Harness

โมเดลคือก้อนพลังแห่งสติปัญญา... ระบบ Harness คือสิ่งที่เปลี่ยนสติปัญญานั้นให้กลายเป็นผลลัพธ์ที่จับต้องได้
เริ่มสร้างสถาปัตยกรรมล้อมรอบความฉลาดของคุณได้แล้วตั้งแต่วันนี้